Memory Traces — memory off vs memory on

Captured: live model (Gemini, `gemini-3.8-flash`, provider "Gemini-Resolv")

Entry point: `generateAssistantReply()` in `resolv-hq-backend/src/lib/ai.ts`, called directly with the same query, knowledge passage, open request and account, changing only the `customerMemory` argument.

What this shows, and what it does not

Shows: with one remembered fact supplied, the live model's escalation draft and reply change to reflect it. In both runs the output is a *draft that has not been submitted*.

Does not show:

- The full chat route (`POST /chat/conversations/:id/messages`). That route also records the run and creates the pending `agent_approvals` row. I did not write to the production database for this trace, so no approval row exists for these runs. The approval gate is unchanged code and was not exercised here.
- A statistical effect. This is one pair of runs on one model; model output varies between runs.
- Loading from the database. The fact was passed in directly. Loader behaviour (owner, master switch, per-fact flag) is covered by `resolv-hq-backend/tests/memory.test.ts`.

Setup

Item	Value
:--	:--
Query	"My internet has been down for three days and nobody has helped me. Please escalate this to a person, and tell me how they will reach me."
Knowledge passage	"Support Escalation Policy" p.1: a support agent reviews escalations and contacts the customer using the contact preferences on file; escalations are not completed automatically and a human must approve them
Open request	"Internet outage" (open)
Caller	Customer, Kampala, Uganda (synthetic account object)
Memory fact (ON run only)	`contact_channel = "SMS only, weekdays after 5pm. Never call."`

Run A — memory OFF

- **Plan:** call `search_knowledge_base`, `draft_escalation_ticket`.
- **Act/Observe:** knowledge search returned the policy passage; draft tool returned a not-submitted draft.
- **Draft key facts:** internet down three days; existing open request; customer reports no assistance.
- **Suggested action:** "Prioritize review of the existing internet outage ticket and contact customer directly to resolve."

- **Priority:** normal.
- **Reply (excerpt):** "...it has not been officially filed yet. According to the Support Escalation Policy, page 1, a support agent ... will reach out to you using the contact preferences on file for your account."
- **Outcome:** escalated. The reply cannot say *how* the person will be contacted.

Run B — memory ON

- **Plan:** call `draft_escalation_ticket` (the model did not search the knowledge base this time).
- **Draft key facts:** internet down three days; existing open request; **Preferred contact: SMS only, weekdays after 5pm (no calls).**
- **Suggested action:** "Review open outage request and **contact customer via SMS after 5pm on weekdays.**"
- **Priority:** high.
- **Reply (excerpt):** "...I have prepared a draft escalation for our team to review. Please note that this draft has not been submitted yet... Based on your contact preferences, our team will reach out to you by SMS on weekdays after 5:00 PM."
- **Outcome:** escalated. The reply answers the "how will they reach me" question using the remembered fact.

Observations

1. **Memory changed the content in the intended way:** the contact preference appears in the draft's key facts, the suggested action and the reply.
2. **Memory did not authorize anything.** Both runs ended in a draft marked not submitted; the reply in run B still says a team member must review it. The memory text contains no permission to act, and the draft tool is read-only.
3. **Differences not attributable to memory.** Run B skipped the knowledge-base search and set priority to high, while run A used normal. The fact says nothing about urgency, so this is most likely ordinary model variance. Do not claim memory changed the priority.
4. **The reply in run B states the contact plan as a commitment** ("our team will reach out to you by SMS..."). The preference is a customer wish, not something staff have agreed to. This is worth a prompt tweak ("will try to contact you by...") and a boundary-matrix case in Week 7.

Capture notes (failed attempts, kept for honesty)

- A first attempt to run both cases back to back succeeded for run A but run B hit Gemini's free-tier limit (5 requests per minute). The gateway then tried Anthropic (`400 credit balance is too low`) and OpenAI (`404 model gpt-4 does not exist`) and fell back to keyword search, returning a generic "couldn't find an exact match" message. That output is a Week 5-style failure/recovery, not a memory result, and is not used above.
- Run B was re-run alone after the quota window. Run A is from the first attempt.
- The Anthropic and OpenAI provider rows are registered as active but not usable (no credit; model id invalid). They need fixing before Week 7's 30-scenario set.